

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I_N.SANDIYANITTIANANDAM(192571029)_ __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A. Midhuna

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Set 2

1. Industrial Robot Automation System

A manufacturing company is developing an industrial robot control system where robotic arms perform assembly operations automatically. The compiler used in the embedded software processes declarations, assignment statements, and Boolean expressions to coordinate robotic movement and safety checks. During production, delays occurred due to inefficient handling of procedure calls and intermediate code generation. The company plans to redesign the compiler to improve execution speed and synchronization between robotic units.

Tasks:

ANSWER:

i)Design intermediate code for robotic movement and safety validation logic. (K3)

Three Address Code (TAC):

t1 = position == target

if t1 goto L1

goto L2

L1:

t2 = safety_sensor == ON

if t2 goto L3

goto L4

L3:

motor = ON

arm = MOVE

goto L5

L4:

motor = OFF

arm = STOP

alarm = ON

L5:

status = COMPLETE

Here, t1 and t2 are temporary variables, while L1–L5 are labels used for control flow. The code checks whether the robotic arm has reached the target and whether the safety sensor is active before allowing movement.

(ii) Explain how procedure calls improve modularity in industrial automation systems. (K4)

Procedure calls divide a large robot-control program into small, reusable modules. For example, separate procedures can be created for MoveArm (), CheckSafety (), PickObject (), and PlaceObject ().

This improves modularity because:

- Each procedure performs a specific task.
- The same procedure can be reused for different robotic operations.
- Changes in one procedure do not require changing the entire program.
- Procedures make the program easier to test and maintain.

- Procedure calls reduce code duplication and improve synchronization between robotic units.

Thus, procedure calls make industrial automation software organized, reusable, maintainable, and efficient.

iii) Analyze the role of compiler optimization in improving manufacturing efficiency. (K5)

Compiler optimization improves the efficiency of embedded robot-control programs by reducing execution time, memory usage, and unnecessary instructions. Techniques such as constant folding, dead-code elimination, common subexpression elimination, and efficient register allocation help generate faster intermediate and machine code.

For industrial robots, faster code execution results in quicker movement decisions, better synchronization, reduced processing delays, and improved production speed. Optimization also reduces processor and memory requirements, which can lower system resource usage.

Therefore, compiler optimization helps achieve faster, reliable, and efficient manufacturing operations.

2. Cybersecurity Intrusion Detection System

A cybersecurity firm is developing an intrusion detection system that continuously monitors network traffic for suspicious activities. The compiler processes Boolean expressions, assignment statements, and case statements to identify different categories of cyberattacks. During real-time monitoring, inefficient intermediate code generation increased system response time and delayed threat detection. The development

team intends to enhance compiler efficiency for faster and more reliable security analysis.

Tasks:

ANSWER:

(i) Generate intermediate representation for network threat detection logic. (K3)

t1 = packets > 1000

if t1 goto L1

goto L2

L1:

t2 = suspicious == 1

if t2 goto L3

goto L2

L3:

threat = DETECTED

alert = ON

goto L4

L2:

threat = NORMAL

alert = OFF

L4:

status = COMPLETE

Here, t1 and t2 are temporary variables used to evaluate Boolean conditions. L1–L4 are labels used for controlling the flow of execution. If

the packet count is high and the traffic is suspicious, the system detects a threat and activates an alert.

ii) Discuss how case statements can efficiently classify multiple cyberattack scenarios. (K4)

Case statements provide a simple and efficient way to classify different types of cyberattacks based on a single value. For example:

```
case attack_type of
```

```
1 threat = "DDoS"
```

```
2: threat = "Malware"
```

```
3: threat = "Phishing"
```

```
4: threat = "Brute Force"
```

```
default: threat = "Unknown"
```

Each case represents a different attack category. This avoids writing many nested if-else statements and makes the program easier to understand and maintain. The compiler can also implement case statements efficiently using jump tables or conditional branches.

(iii) Evaluate the importance of compiler optimization in real-time cybersecurity applications. (K5)

Compiler optimization improves the execution speed of intrusion detection software by reducing unnecessary instructions and memory usage. Techniques such as constant folding, dead-code elimination, common subexpression elimination, and efficient register allocation generate faster code.

In real-time cybersecurity, faster execution allows network traffic to be analyzed quickly, reducing threat detection delay and response time. It also improves processor and memory utilization.

Therefore, compiler optimization is important for building fast, reliable, and responsive real-time intrusion detection systems.

3. Digital Payment Gateway System

A fintech company is building a digital payment gateway to process online transactions securely and efficiently. The compiler used in the transaction engine must evaluate Boolean conditions, assignment statements, and procedure calls during payment verification and fraud detection. During peak transaction periods, improper backpatching caused delays in payment confirmation and rollback operations. The organization plans to redesign the compiler to improve transaction reliability and scalability.

Tasks:

ANSWER:

i)Construct intermediate code for payment verification and fraud detection logic. (K3)

Three Address Code (TAC):

```
t1 = payment_valid == 1
```

```
if t1 goto L1
```

```
goto L2
```

```
L1:
```

```
t2 = fraud_score > 70
```

```
if t2 goto L3
```

```
goto L4
```

L3:

transaction = BLOCKED

rollback = ON

goto L5

L4:

transaction = APPROVED

confirmation = ON

L2:

transaction = REJECTED

confirmation = OFF

L5:

status = COMPLETE

Here, t1 and t2 are temporary variables used to evaluate conditions. L1–L5 are labels used for controlling the flow. If the payment is valid and the fraud score is high, the transaction is blocked and rollback is activated. Otherwise, the payment is approved.

(ii) Explain how backpatching supports conditional transaction processing. (K4)

Backpatching is a technique used in intermediate code generation to fill in the correct target addresses of jump instructions when the target is not yet known.

For example:

if payment_valid goto _

goto _

Later, the compiler fills the blanks with the appropriate labels:

```
if payment_valid goto L1
```

```
goto L2
```

In payment processing, backpatching helps connect conditions such as payment verification, fraud detection, approval, rejection, and rollback correctly. It supports efficient control flow without requiring all target labels to be known immediately.

(iii) Analyze the role of compiler design in ensuring secure and scalable fintech applications. (K5)

Compiler design plays an important role in generating efficient, reliable, and secure code for fintech applications. Proper intermediate code generation and optimization reduce execution time and resource usage.

Security-related checks such as authentication, payment validation, fraud detection, and transaction verification can be efficiently implemented through Boolean expressions, conditional statements, and procedure calls. Compiler optimization also improves performance during peak transaction loads.

Therefore, effective compiler design helps fintech systems achieve faster transaction processing, reliable rollback, better resource utilization, security, and scalability.

4. Smart Agriculture Monitoring System

An agritech company is developing a smart agriculture platform that monitors soil moisture, temperature, and irrigation systems automatically.

The compiler in the embedded controller processes declarations, assignment statements, and Boolean expressions to activate irrigation based on environmental conditions. Inefficient handling of intermediate code caused delays in irrigation control during critical farming conditions. The software team aims to improve compiler performance for real-time agricultural automation.

ANSWER:

(i) Develop intermediate code for irrigation control and environmental monitoring. (K3)

t1 = soil_moisture < 30

if t1 goto L1

goto L2

L1:

t2 = temperature > 35

if t2 goto L3

goto L2

L3:

pump = ON

irrigation = START

goto L4

L2:

pump = OFF

irrigation = STOP

L4:

status = COMPLETE

Here, t1 and t2 are temporary variables used to evaluate conditions. L1–L4 are labels used for controlling the flow of execution. If the soil moisture is low and the temperature is high, irrigation is started. Otherwise, the irrigation system remains OFF.

(ii) Explain the role of Boolean expressions in automated farming systems. (K4)

Boolean expressions are used to make decisions based on environmental conditions. For example, conditions such as `soil_moisture < 30` and `temperature > 35` are evaluated as TRUE or FALSE.

They help the system automatically decide whether to:

- Start or stop irrigation.
- Activate water pumps.
- Monitor critical temperature conditions.
- Perform actions without human intervention.

Thus, Boolean expressions provide automatic and condition-based control in smart farming systems.

(iii) Evaluate how compiler optimization enhances efficiency in smart agriculture applications. (K5)

Compiler optimization improves the execution speed of embedded agricultural control programs by removing unnecessary instructions and reducing memory usage. Techniques such as constant folding, dead-code elimination, common subexpression elimination, and efficient register allocation help generate faster code.

Faster code execution allows the irrigation controller to respond quickly to changes in soil moisture and temperature. It also reduces processor usage and improves energy efficiency.

Therefore, compiler optimization provides faster, reliable, and efficient real-time irrigation and environmental monitoring in smart agriculture.

5. Cloud-Based Video Streaming Platform

A multimedia company is developing a cloud-based video streaming platform that dynamically adjusts video quality based on network conditions. The compiler used in the streaming engine processes case statements, assignment statements, and procedure calls for resource allocation and bandwidth management. During high user traffic, inefficient intermediate code generation caused buffering delays and reduced streaming quality. The company plans to redesign the compiler to improve scalability and performance.

ANSWER:

i) Design intermediate code for bandwidth allocation and quality adaptation logic. (K3)

t1 = bandwidth < 2

if t1 goto L1

goto L2

L1:

quality = LOW

bandwidth_alloc = 1

goto L4

L2:

t2 = bandwidth < 5

if t2 goto L3

goto L5

L3:

quality = MEDIUM

bandwidth_alloc = 3

goto L4

L5:

quality = HIGH

bandwidth_alloc = 5

L4:

status = COMPLETE

(ii) Explain how case statements help manage multiple streaming quality levels. (K4)

Case statements allow the streaming system to select different video quality levels based on a single value such as bandwidth category.

case bandwidth_level of

1: quality = "LOW"

2: quality = "MEDIUM"

3: quality = "HIGH"

default: quality = "AUTO"

Each case represents a different streaming quality. This is simpler than using many if-else statements and makes the program easier to understand and maintain.

Thus, case statements provide **fast and organized management of multiple video quality levels.**

(iii) Analyze the significance of compiler optimization in cloud-based multimedia systems. (K5)

Compiler optimization improves the performance of streaming software by reducing execution time, memory usage, and unnecessary instructions. Techniques such as dead-code elimination, constant folding, common subexpression elimination, and efficient register allocation help generate efficient code.

In cloud-based streaming, optimized code enables faster bandwidth allocation, quality adaptation, resource management, and request processing. It also helps handle a large number of users with better resource utilization and reduces buffering delays.

Therefore, compiler optimization is important for achieving scalable, fast, and smooth video streaming performance.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation